Міністерство освіти і науки України

Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Проєктування, моделювання та забезпечення якості програмного забезпечення систем»

Робота «Завдання для самоперевірки знань за офлайн лекцією 24.10.2026»

Тема «Проєктування NoSQL-архітектури для платформи Smart City»

Виконав варіант 19

Студент КНТ-116м Онищенко О.А.

Прийняли

Викладач Табунщик Г.В.

2026

МЕТА

Перевірити вміння підбирати моделі даних NoSQL (Key-Value, Document, Column-family, Graph) під задачі Smart City, аналізувати компроміси CAP/PACELC та BASE vs ACID, а також проєктувати денормалізовані структури даних без JOIN.

ЗАВДАННЯ

Завдання 1 — Архітектурний вибір NoSQL-моделей.

Для кожного з 4-х модулів оберіть відповідну модель NoSQL (Key-Value, Document, Column-family, Graph) та запропонуйте конкретну СУБД із матеріалів презентації (наприклад: Redis, MongoDB, Cassandra, Neo4j). Обґрунтуйте вибір.

Завдання 2 — Аналіз CAP / PACELC та ACID / BASE:

Визначте для Модуля 1 (GPS-моніторинг) та Модуля 3 (Реєстр жителів) їхню позицію в теоремі CAP (CP чи AP). Чому обрано саме такий компроміс? Поясніть, чому для Модуля 1 підходить концепція BASE (Eventual Consistency), а для фінансових нарахувань у Модулі 3 потрібна сувора узгодженість ACID.

Завдання 3 — Проєктування JSON-схеми (Schema Design):

Складіть JSON-документ для Модуля 3 (Картка жителя Smart City). Схема має показувати, як в одному документі зберегти базову інформацію про жителя, його пільги та останні 2 звернення до міської служби (без використання реляційних JOIN).

ВИКОНАННЯ

1 Архітектурний вибір NoSQL моделей

Обрані модулі та їх обґрунтування наведено на таблиці 1.1.

Назва модулю	**Обрана модель**	**СКБД**	**Обґрунтування**
Моніторингу громадського транспорту	Document	MongoDB	Звичайні документи підійдуть найкраще
Міського графа та маршрутизації	Graph	Neo4j	Бо інформація зберігається у графах
Реєстру жителів та муніципальних послуг	Column-family	Cassandra	Потрібне групування, тому family groups тут буде корисним
Оперативного кешування авторизацій та сесій	Key-Value	Redis	Для кешу зазвичай використовують key-value pairs

2 Аналіз CAP / PACELC та ACID / BASE

2.1 CAP / PACELC

CP (Consistency + Partition tolerance) – точність важливіша за те, щоб відповідати завжди.

AP (Availability + Partition tolerance) – краще відповісти, навіть якщо трохи не свіжо.

Модуль GPS-моніторингу краще підходить під AP, бо такі дані потребують високого часу безвідмовної роботи.

Модуль реєстру жителів краще підходить під CP, бо для таких даних важливіша точність.

2.2 ACID / BASE

ACID – точність понад усе. Або все проходить, або нічого.

BASE – відповісти понад усе. Відповісти зараз, навіть якщо дані не зовсім точні.

Для фінансових нарахувань важлива точність, тому ACID підходить краще.

Для моніторингу громадського транспорту важлива безперебійна робота, тому BASE тут підходить краще.

3 Проєктування JSON-схеми

JSON-схему наведено нижче:

```
JSON-Schema
├── User_ID
├── Full_Name
├── Address
│   ├── Street_Name
│   └── House_Number
├── Benefits (пільги)
│   ├── Benefit_Type
│   └── Description
└── Recent_Requests
    ├── Request_ID
    ├── Service_Name
    └── Request_Date
```

Останні 2 звернення можна сформувати на основі поля Recent_Requests, витягнувши з нього останні 2 записи.

Прикладом об’єкту JSON за зазначеною схемою є наступний:

```
[
  {
    "User_ID": 1,
    "Full_Name": "Маслюк Петро",
    "Address": {
      "Street_Name": "Перемоги",
      "House_Number": 12
    },
    "Benefits": [
      {
        "Benefit_Type": "пільговий проїзд",
        "Description": "дозволяє безкоштовний проїхд на громадському
транспорті"
      }
    ],
    "Recents_Requests": [
      {
```

```
        "Request_ID": 1,
        "Service_Name": "субсидії для житла",
        "Request_Date": "2026-10-25"
      }
    ]
  }
]
```